

CHECK WHAT YOU LEARN'T

Check Yourself - Unit 1**A. Choose the correct answer**

- Ohm's law is written as
(a) $V = I + R$ (b) $V = I \times R$ (c) $V = I / R$ (d) $I = V \times R$
- A 220 ohm resistor with an LED lets through roughly
(a) 1.4 mA (b) 13.6 mA (c) 136 mA (d) 1.36 A
- In the middle of a breadboard, holes are joined in
(a) columns of five (b) rows of ten (c) no pattern (d) pairs
- The DHT11 sends its reading as
(a) a voltage (b) a coded sequence of pulses (c) a resistance (d) a colour
- A push switch needs a 10K resistor in order to
(a) limit current (b) hold the input at a known level (c) make it faster (d) protect the LED
- A system differs from a gadget because it
(a) is larger (b) has several parts working together (c) costs more (d) uses a battery

B. Fill in the blanks

- The push that moves electricity along is called _____.
- Analog pins on the Arduino Uno are labelled _____ to _____.
- A motor must never be connected directly to an _____ pin.
- Flame testing is done only with a supervised _____ in a metal tray.

C. Answer in one or two lines

- Name one safety system that requires two senses to agree before it acts.
- Work out the current through a 330 ohm resistor with 3 volts across it.
- Explain why the LDR module offers both an AO and a DO pin.
- Your LED does not glow. Write the first three things you would check.

UNIT 2**The Arduino IDE**

Sessions 3, 4 and 5

Most people use an Arduino for years without ever asking what is actually happening inside it. This unit answers that question, because once you can picture the chip, the ADC and the upload process, debugging stops being guesswork.

By the end of this unit you will be able to:

- Name the main parts of the Arduino Uno board and say what each does.
- Describe what happens between pressing Upload and the program running.
- Explain what `analogRead()` actually measures and what its resolution is.
- Install and use an external library.

SESSION 3

one class

Inside the Board and Inside the Chip

Today you will

- Identify the main components on the Arduino Uno board.
- Learn what the ATmega328P contains.
- Upload Blink and confirm your whole setup works.

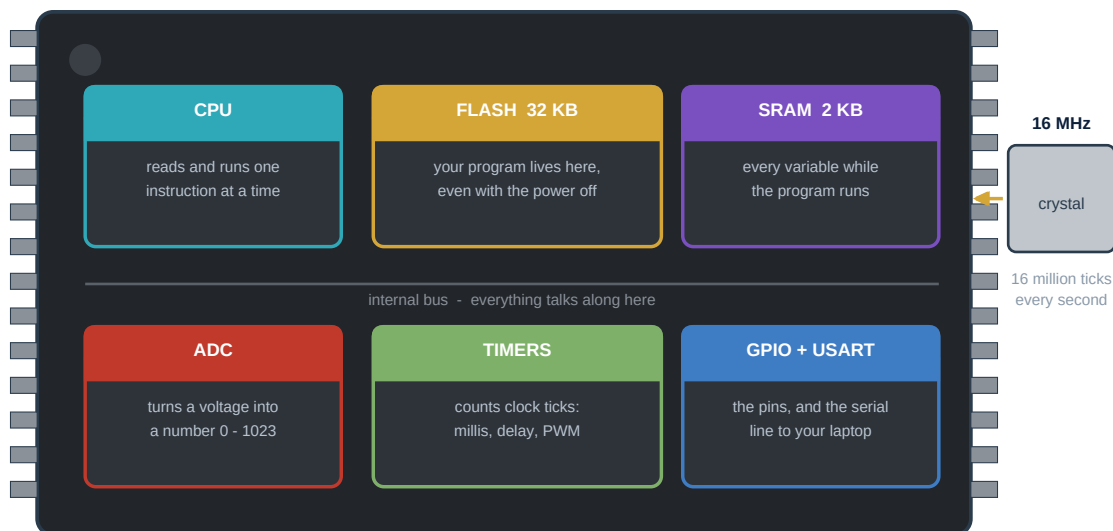
The blue board in your hand is not the computer. It is a **support system** for one chip. Look at it closely and you can find every part of that support system.

What you can see on the board	What it is for
The long black chip in the middle	The ATmega328P - the actual computer
A small silver oblong beside it	A 16 MHz quartz crystal that sets the chip's heartbeat
A second, smaller chip near the USB socket	Translates USB into the simple serial the ATmega understands
A three-legged part near the power jack	A voltage regulator, turning 7 to 12V into a steady 5V
The black barrel socket	Power in from a battery or adapter
Rows of black sockets	The headers - your pins D0 to D13 and A0 to A5
A small button	Reset - restarts your program from the beginning

Behind the Scenes

The ATmega328P costs a fraction of the board's price. Everything else on that PCB exists to make the chip easy for a human to use: to power it safely, to give it a steady clock, and to let a laptop talk to it over USB. A finished commercial product usually drops the board entirely and solders the bare chip onto its own circuit - which is exactly what happens inside a washing machine or a microwave.

ATmega328P - the chip inside the Arduino Uno



Everything you write ends up as instructions the CPU fetches from flash, one at a time, sixteen million times a second.

Figure 2.1 - Behind the blue board: what is actually doing the work

Inside the chip	Size	What it holds
Flash memory	32 KB	Your compiled program. Survives power loss.
SRAM	2 KB	Every variable while the program runs. Lost on power off.

Inside the chip	Size	What it holds
EEPROM	1 KB	Small amounts of data you want kept after power off.
CPU	-	Fetches and executes one instruction at a time.
ADC	10-bit	Turns a voltage into a number from 0 to 1023.
Timers	three	Count clock ticks. These run <code>millis()</code> , <code>delay()</code> and PWM.

Watch Out

Notice how small SRAM is: 2 KB for every variable at once. A single line like `Serial.println("Temperature is now");` stores that whole sentence in SRAM. Programs with many long messages can quietly run out of memory and behave very strangely - and the compiler will not always warn you.

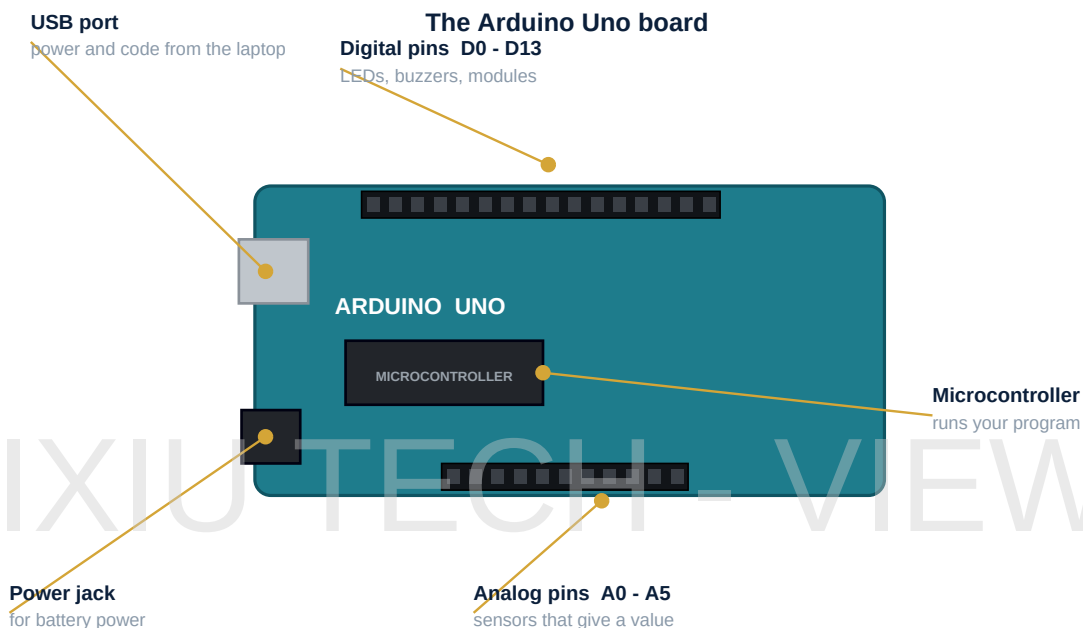


Figure 2.1 - Know your board before you wire anything to it

SESSION 4

one class

How Your Code Reaches the Chip

Today you will

- Follow your program from typed text to running machine code.
- Learn what the bootloader does and why the board resets.
- Read an error message and work out what it means.

When you press Upload, four separate things happen in about two seconds. Most people never learn them, and then cannot explain why an upload failed. You will be able to.

- 1 Compiling.** Your English-like text is translated into machine instructions the ATmega understands. Any spelling or grammar mistake stops the process here, and nothing is sent.
- 2 Linking.** The libraries you included are joined to your code, and the whole thing becomes one file of hexadecimal numbers - the .hex file.
- 3 Resetting.** The IDE briefly pulls the board's reset line. The chip restarts and, for about a second, runs a small program already built into it called the **bootloader**.

- 4 **Writing.** The bootloader listens on the serial line, receives your .hex file, and writes it into flash memory. Then it hands control to your program, which starts running immediately.

Behind the Scenes

That one-second bootloader window explains a fault you will certainly meet. If your program starts talking on the serial line the instant it boots, it can shout over the bootloader and the next upload fails. It also explains why the two small LEDs near the USB socket flicker during upload - that is your program travelling down the wire, one packet at a time.

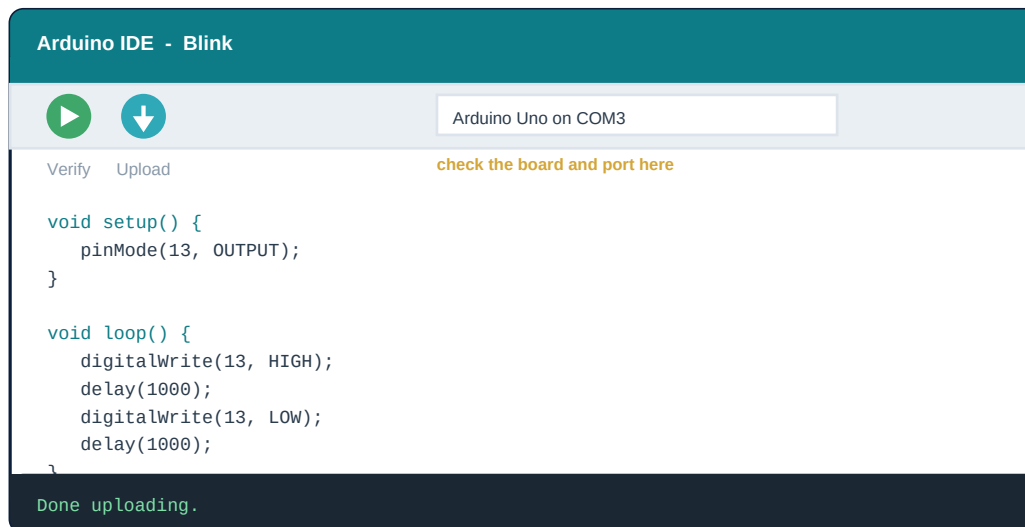


Figure 2.2 - Verify checks your code; Upload sends it to the board

What the screen says	Which stage failed	How to fix it
expected ';' before ...	Compiling	Add a semicolon at the end of the line above
'digitalwrite' was not declared	Compiling	Capitals matter: digitalWrite
no matching function for call to ...	Compiling	Wrong number or type of values passed
Board at COM__ is not available	Resetting	Re-select Tools > Port, or change cable
programmer is not responding	Writing	Wrong board selected, or the bootloader was interrupted
Sketch too big	Linking	Your program exceeds 32 KB of flash
Low memory available	Linking	Too many variables or long strings for 2 KB of SRAM

Blink.ino - the test that proves the whole chain works

```
int led = 13;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
  Serial.println(millis()); // how long the board has been running
}
```

Reading the World: the ADC and Libraries

Today you will

- Understand what `analogRead()` actually does inside the chip.
- Work out the resolution of a 10-bit ADC.
- Install a library and use it.

An analog pin does not measure light, or heat, or distance. It measures exactly one thing: **voltage**. Inside the ATmega there is a circuit called an **Analog to Digital Converter**, and its job is to compare the voltage on the pin against a known reference and report the answer as a whole number.

Analog to Digital Conversion

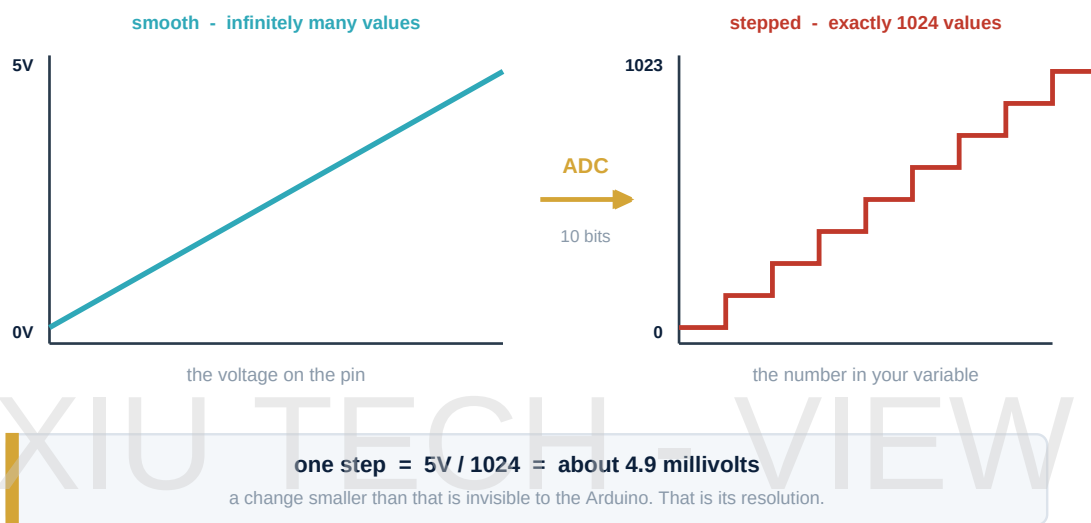


Figure 2.2 - Every `analogRead()` is a measurement rounded to the nearest step

The ATmega's ADC is **10-bit**, which means it reports 2 to the power 10 different values - 1024 of them, numbered 0 to 1023. Spread across a 5 volt range, each step is 5 divided by 1024, or about 4.9 millivolts. That number is the sensor's **resolution**: any change smaller than one step is simply invisible.

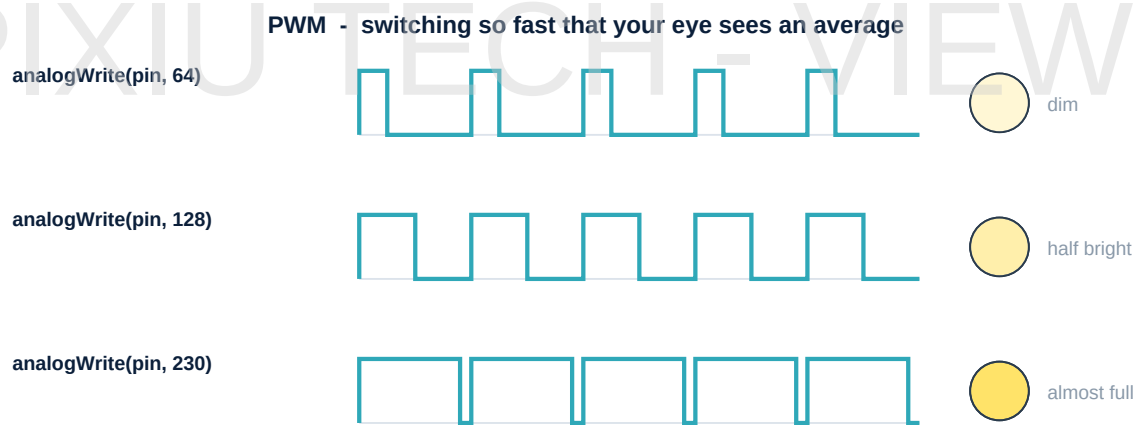
Voltage on the pin	What <code>analogRead()</code> gives	Working
0 V	0	nothing to measure
1.25 V	about 256	$1.25 / 5 \times 1023$
2.5 V	about 512	half of full scale
5 V	1023	full scale

Behind the Scenes

The ADC does not measure instantly. It uses a technique called successive approximation: it guesses half, asks *is the voltage above this?*, then halves the remaining range and asks again, ten times over. Each `analogRead()` therefore takes about 100 microseconds. That is fast, but it is not free - and in a tight loop reading six sensors, it adds up.

5.1 The Opposite Trick: PWM

If the ADC turns a voltage into a number, PWM does something close to the reverse. A digital pin cannot produce half of 5 volts - it is only ever fully on or fully off. So the chip cheats: a timer switches the pin on and off about 490 times a second, and by changing how much of each cycle is spent on, the *average* voltage changes. Your eye, and a motor's inertia, are far too slow to notice the switching.



The pin is still only ON or OFF. It is the amount of time spent ON that changes.

Only pins marked with ~ can do this: D3, D5, D6, D9, D10 and D11.

Only six pins on the Uno are wired to those timers - D3, D5, D6, D9, D10 and D11 - and they are marked with a small ~. You will use PWM this year to set the speed of your robot's motors, where `analogWrite(ena, 130)` means *keep the enable pin on for about half of each cycle*.

5.2 Libraries: Standing on Someone Else's Work

Some components are far too complicated to talk to by hand. The DHT11 sends its reading as a precisely timed sequence of forty pulses. The 16x2 LCD needs a specific start-up ritual before it will show anything at all. Decoding these yourself would take hundreds of lines.

A **library** is a bundle of instructions somebody else has already written and tested. You include it once, and then a single sensible line does the work. This is not cheating - it is how all professional software is built.

- 1 Open **Tools > Manage Libraries** in the IDE.
- 2 Search for **DHT sensor library** and install the one by **Adafruit**.
- 3 If it offers **Adafruit Unified Sensor** as well, accept - it depends on it.
- 4 The LiquidCrystal library for your display is already built in; nothing to install.
- 5 Close the manager. The new instructions are now available to every sketch.

New word	What it means
ATmega328P	The microcontroller chip that is the real computer on the board.
Flash / SRAM	Memory for the program / memory for variables while running.
Bootloader	A small built-in program that receives your code and writes it to flash.
ADC	The circuit that turns a voltage into a number.
Resolution	The smallest change a measuring system can detect.
Library	Ready-made code you include and use.